

Utilizing software for system modelling

ARTURO HERNÁNDEZ MARTÍNEZ

GONZALO MORTE GÓMEZ

Why System Modelling?

- **Visual Content**
 - **Clear communication:** System modelling translates complex client requirements into visual blueprints (like UML diagrams) that developers can easily understand.
 - **Risk reduction:** Creating visual models before writing any code helps us identify missing functionalities and potential errors early in the project.
 - **Stakeholder alignment:** It provides a universal language. Both non-technical clients and technical programmers can look at a Use Case Diagram and agree on how the system should work.
 - **Project efficiency:** Proper planning through modelling saves time, reduces development costs, and ensures the final product matches the initial expectations.

The Cost of Poor Planning

What happens without System Modelling?

- **Scope Creep:** Without clear visual boundaries, the project requirements keep growing uncontrollably.
- **Wasted Budget:** Developers spend hours programming features that the client didn't actually want, leading to expensive rework.
- **Hidden Information:** Information stays in people's heads or scattered emails instead of a centralized, visual document.

Conclusion: Investing time in a modelling tool now prevents catastrophic failures during the development phase later.

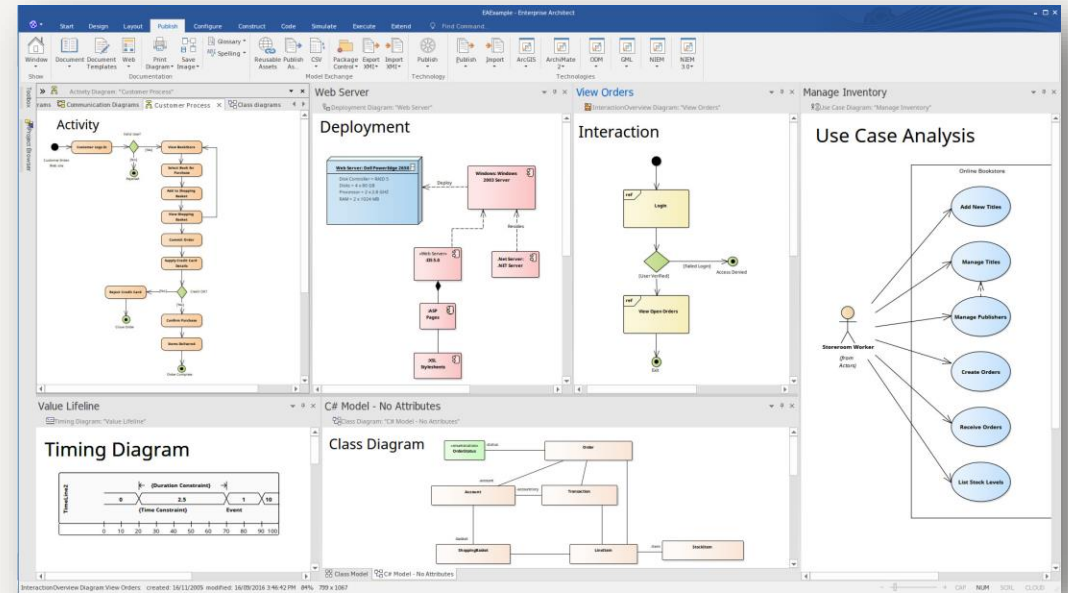


User Needs

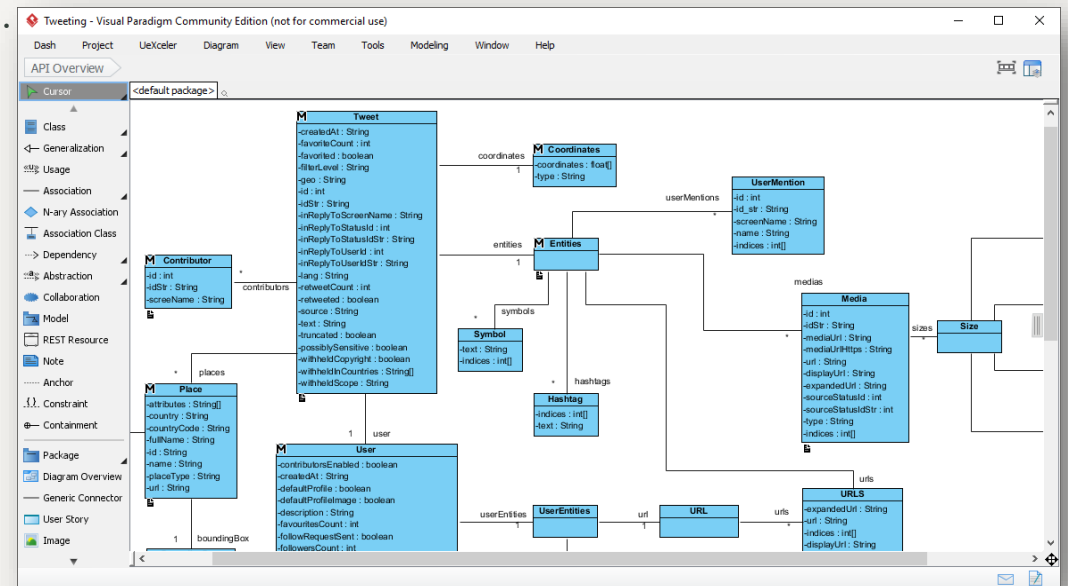
- **Cloud-based accessibility:** The team requires a web application. We want to avoid complex desktop installations and software compatibility issues.
- **Real-time collaboration:** Since our team members work simultaneously, the software must support live co-authoring, similar to Google Docs.
- **Intuitive interface:** We need a drag-and-drop system. The learning curve must be minimal so non-technical stakeholders can use it immediately.
- **Exporting capabilities:** The tool must allow us to export diagrams in formats like PDF or PNG for our official project documentation.
- **Budget constraints:** As a Project Manager, I need a tool that offers a robust free tier for basic tasks and affordable premium plans for future scaling.

Market Analysis

- **Option 1: Enterprise Architect (Sparx Systems)**
- *Type:* Standalone Desktop Application.
- *Price:* From \$245 (One-time license).
- *Verdict:* Extremely powerful for massive enterprise systems, but too expensive and complex for our current agile team needs.

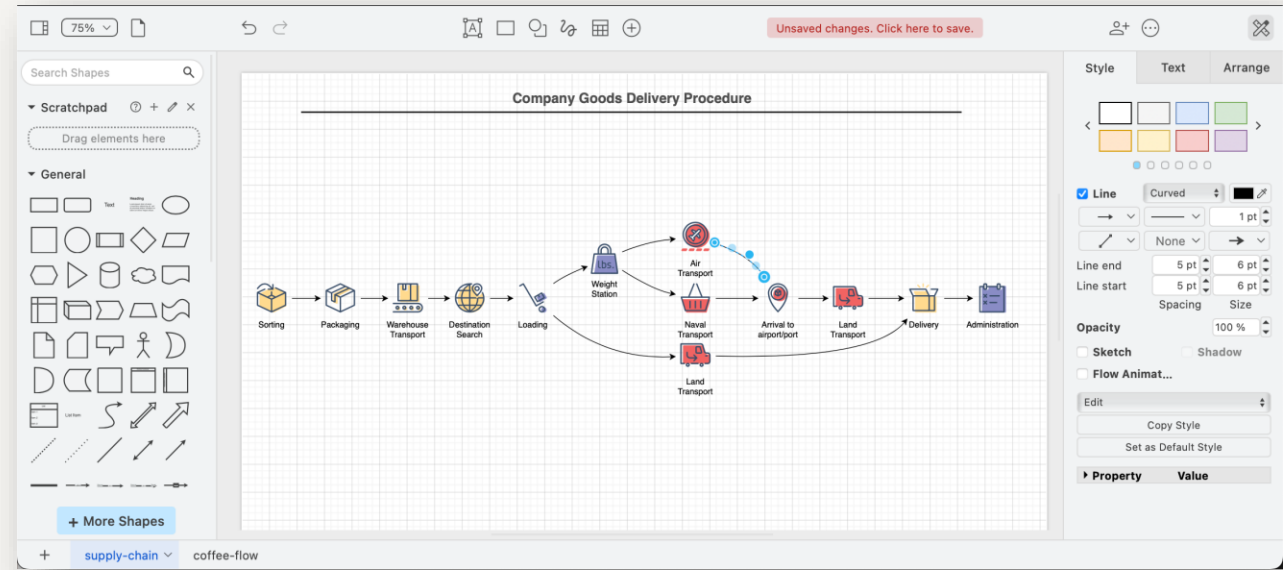


- **Option 2: Visual Paradigm (Visual Paradigm)**
- *Type:* Web Application & Desktop.
- *Price:* Free basic tier / \$4 per month for Starter plan.
- *Verdict:* Great feature set, but the interface can feel a bit cluttered for beginners.

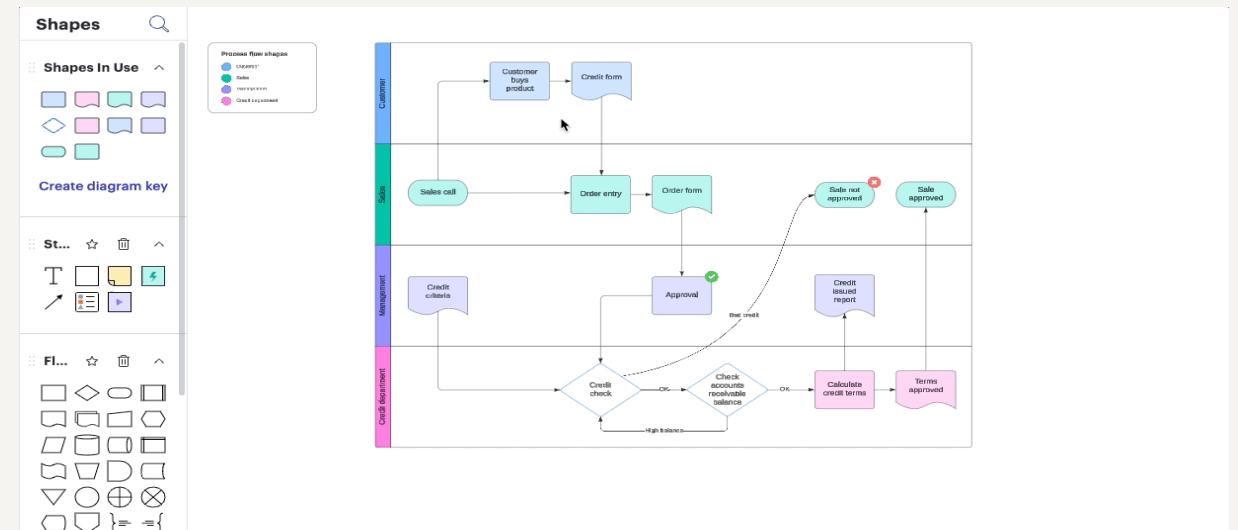


Market Analysis

- **Option 3: Draw.io (JGraph)**
- *Type:* Web Application.
- *Price:* 100% Free (Open source).
- *PM Verdict:* Excellent free tool, but it lacks advanced team management features and real-time collaboration can be buggy.



- **Option 4: (Lucid Software Inc.)**
- *Type:* Web Application.
- *Price:* Free basic tier / \$7.95 per month for Individual Premium.
- *PM Verdict:* A highly polished interface with flawless real-time collaboration and cloud storage integration.

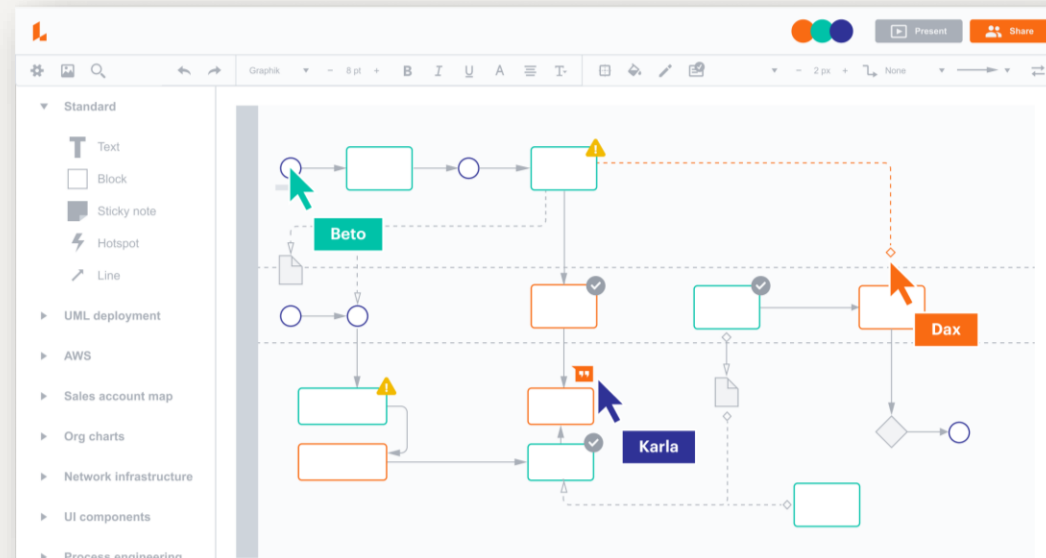


Comparative Price Analysis

Program Name	Type	Producer	Price (USD)
Enterprise Architect	Standalone Desktop	Sparx Systems	From \$245 (One-time)
Draw.io	Web Application	JGraph	100% Free
Visual Paradigm	Web Application	Visual Paradigm	Free basic / \$4/month
Lucidchart	Web Application	Lucid Software Inc.	Free basic / \$7.95/month

Motivation for Our Final Choice

- **The Winning Tool:** We have officially selected **Lucidchart** for this project.
- **Best Balance of Features and Price:** It provides the most professional interface and the smoothest real-time collaboration compared to the free alternatives.
- **Extensive Template Library:** It includes hundreds of pre-built UML and system architecture templates, which will drastically speed up our workflow.
- **Zero Cost for Immediate Needs:** The free tier allows us to create up to 3 editable documents with basic shapes, which is exactly what we need to start modelling today without impacting the budget.



The PM Perspective

Specifying Needs vs. Buying Products

- **Requirements First:** As a Project Manager, I specify functional expectations based on project needs, not based on what a specific software offers.
- **Risk Mitigation:** System modeling is a tool for **risk reduction**; we identify missing functionalities before a single line of code is written.
- **Stakeholder Alignment:** The spec must ensure a "universal language" so that non-technical clients and programmers remain aligned on system behavior.
- **Efficiency:** Proper planning through these specifications reduces development costs and prevents **Scope Creep**.

Prioritization Framework

The MoSCoW System

To manage our modeling project effectively, we categorize every functionality using the **MoSCoW** system:

- **Must have:** Critical requirements; without these, the modeling process is impossible.
- **Should have:** Important requirements that are not vital but add significant value.
- **Could have:** Desirable features that increase efficiency if the budget/time allows.
- **Won't have:** Features that are not a priority for this specific project phase.

Functional Specifications

Core Modeling Engine

Functionality Name	Purpose	Input Information	Output Information	MoSCoW
Standardized UML Library	Ensure models follow industry-standard syntax.	Selection of UML 2.5 shape categories (Class, Sequence, etc.).	A validated visual toolbox of compliant symbols.	Must Have
Intelligent Relationship Anchoring	Maintain logic when moving system components.	Dragging a connector line between two system entities.	Dynamic, non-breaking logical links between objects.	Must Have
Layered Architecture View	Separate complex systems into manageable "views".	User creates a new "Layer" and assigns specific shapes.	A multi-dimensional diagram that can hide/show complexity.	Should Have

Functional Specifications

Collaboration & Control

Functionality Name	Purpose	Input Information	Output Information	MoSCoW
Real-time Co-Authoring	Allow simultaneous modeling by the architect team.	Simultaneous mouse/keyboard inputs from multiple users.	A synchronized, live-updating canvas for all users.	Should Have
Audit Trail / Revision History	Track who changed the system design to prevent errors.	Automatic logging of every shape modification/deletion .	A chronological list of versions with restore points.	Should Have
Contextual Commenting	Centralize feedback within the model to avoid "siloes of info".	Highlighting a component and typing a text string.	Persistent anchored notes visible to all stakeholders.	Could Have

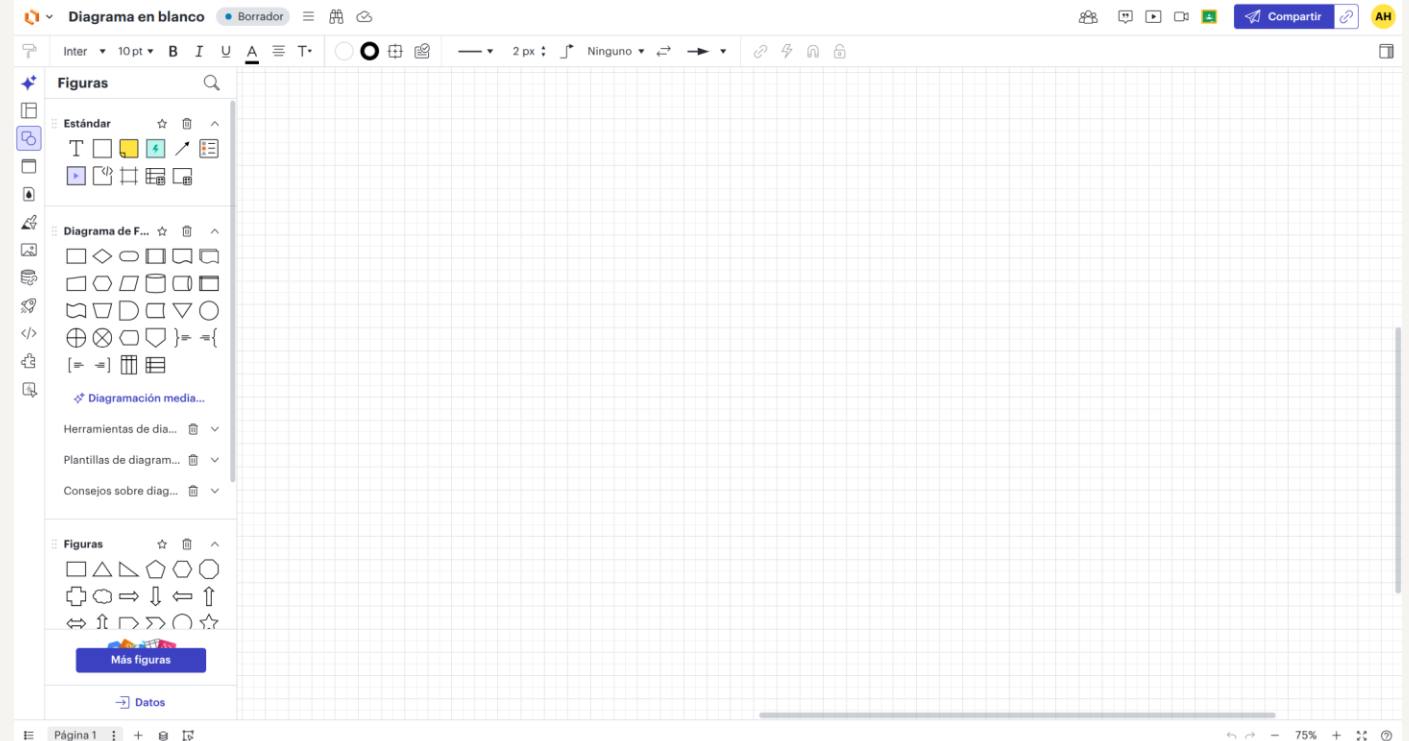
Functional Specifications

Documentation & Integration

Functionality Name	Purpose	Input Information	Output Information	MoSCoW
Multi-format Vector Export	Provide blueprints for official documentation.	User selects export command and file type (PDF/PNG/SVG).	High-resolution files suitable for project reports.	Must Have
Database Schema Import	Automatically generate ERDs from existing code.	Uploading a SQL DDL file or connecting a live database.	An automatically rendered Entity Relationship Diagram.	Could Have
Cloud Storage Sync	Ensure 24/7 access and prevent local file loss.	Saving the document; linking to Google Drive/OneDrive.	A secure, hosted file accessible via web browser.	Must Have

Exercise: Getting Started with Lucidchart

- Go to: www.lucidchart.com
- Click on: "Sign up for free" (Top right corner).
- Create: Click on "+ New" > "Lucidchart" > "Blank Document".



Exercise: System Modeling for an Online Store

- **Task 1: Basic Structure & Actors**
 - Create a **UML System Boundary** box and title it *"Library Management System"*.
 - Add three Actors outside the box: *"Student"* (Left), *"Librarian"* (Right), and *"Database System"* (Right).
 - Inside the box, create four Use Cases (ovals): *"Search Book"*, *"Borrow Book"*, *"Return Book"*, and *"Update Catalog"*.
- **Task 2: Connections & Formatting**
 - Draw solid lines connecting the *"Student"* to *"Search Book"*, *"Borrow Book"*, and *"Return Book"*.
 - Connect the *"Librarian"* to *"Update Catalog"*.
 - Connect all four Use Cases to the *"Database System"* actor.
 - Change the color of all Actor figures to **Blue** and all Use Case ovals to **Yellow**.
- **Task 3: Your own system modelling**
 - **Scenario:** Think of an app or system you use every day (e.g., Spotify, Uber, a video game menu, or a smart home system). Model it from scratch!
 - **Minimum Requirements:**
 - **1 System Boundary** with the clear name of your chosen system.
 - **Minimum 2 Actors** (e.g., User, Server, Driver, Admin).
 - **Minimum 5 Use Cases** (ovals representing the main actions).
 - All Actors must be properly connected to their specific actions using lines.
 - Use at least **two different colors** to organize your diagram.